Revenue Enablement

Copilot CLI | Job aid

What is it

GitHub Copilot CLI is a GitHub-native agent for the terminal, built to plan, execute, refactor, test, and ship software. It allows developers to parallelize work across agent fleets and preserve context across sessions, making it possible to move from `/plan` to pull request without rebuilding state.

Now **generally available**, it is ready for broader enterprise rollout under GitHub's standard GA terms.

- **Who benefits:** All GitHub Copilot customers. Copilot CLI is included in all plans at no additional cost (Free, Pro, Pro+, Business, Enterprise).
- **Simple setup:** Developers install Copilot CLI, authenticate with their GitHub Copilot-enabled account, and use natural language commands in the terminal. Admins can enable it org/enterprise-wide for Business and Enterprise plans.

Feature overview

- **Terminal-native Copilot experience** that works where developers run builds, tests, git operations, scripts, and automation.
- **Agentic assistance** that can help with multi-step tasks like debugging, refactoring, and implementing changes.
- **Key GA narrative anchors:**
    - **Orchestrate parallel subagents:** Move beyond one-off assistance by using `/fleet` to execute in parallel or run multiple models at once. This

allows teams to turn complex, manual work into repeatable agentic workflows that scale expertise across the org.

- **Persistent context:** Reduce the "re-explanation tax" by maintaining intent across multi-step tasks. Copilot remembers your decisions and goals, acting as a unified bridge so you can pick up where you left off without rebuilding context from scratch.
- **Move quickly between terminal and editor** by starting work in the CLI and hand-off work to the editor with `/IDE`.
- **Multi-provider choice:** Access the latest models from frontier providers—including Anthropic, Google, and OpenAI. This ensures you always have the best tool for the specific task and aren't locked into a single provider as the industry evolves.
- **Preview-before-execution** experience helps keep developers in control.
- **Extensible** via open standards (including MCP-powered extensibility) to connect custom skills and plugins where appropriate.

Problems we're solving

Developers need AI that works across tools, maintains context, and scales complex work, not just one-off suggestions.

Before vs. after

Before: AI assistance was not available in the terminal. Developers had to switch to an IDE or browser to use AI.

After: AI assistance is now available directly in the CLI, so developers can stay in the terminal while working

So what: Why it matters

- **Keeps developers in flow:** Copilot CLI holds persistent context, reducing time lost to re-explaining intent within the CLI across multi-step tasks.
- **Scales agentic work beyond the IDE:** Brings agentic workflows into terminal to accelerate debugging, refactoring, build/test loops, and automation.
- **Supports choice without fragmentation:** Access the latest models from across the industry (OpenAI, Anthropic, Google) through a single, governed interface. You gain the flexibility of the entire AI ecosystem while standardizing on GitHub as your core workflow surface.
- **Scales engineering expertise:** The `/fleet` capability allows teams to orchestrate parallel subagents for complex tasks. By turning high-intensity manual work into repeatable agentic workflows, you improve consistency and ensure your most effective patterns are accessible to the entire team.
- **GA unlocks confident rollout:** Admins can adopt and scale Copilot CLI under standard GitHub GA terms.
- **Customer testimonials:**
    - "Copilot CLI fits naturally into how our engineers already work. For teams that live in the terminal, it's faster and less distracting than switching between tools. It feels like a dependable sidekick in the command line, helping developers move through everyday tasks with less friction and more focus." Abhishek Chaturvedi, Senior Principal Engineer, Docusign
    - "Copilot CLI helps our developers focus on the work that matters most and increases developer velocity. By bringing Copilot directly into the terminal, teams can move from intent to execution faster." -Daniel Evans, Director of Software Engineering, London Stock Exchange

Now what: Seller guidance

Who to target

Primary: Admins

Titles: GitHub Org/Enterprise Admin
Focus: Emphasize that Copilot CLI is included with Copilot Business and Enterprise, no additional billing, API keys, setup, or policy configuration required. Customers simply enable it in their enterprise settings.

Secondary: Developers & CLI-Heavy Engineers
Titles: Software Engineer, Senior/Staff Engineer, DevOps, Platform Engineer, SRE
Focus: Reducing the "context switch tax" in Git, Docker, and CI/CD loops.

Tertiary: Engineering & Platform Leaders
Titles: Director/VP Eng, Head of Platform, DevEx, DevOps Manager
Focus: Closing the "execution gap" and driving platform-wide AI adoption.

Enterprise Context: CTO/CIO
CLI supports platform-level conversations around:

- Copilot adoption depth
- Workflow expansion
- AI embedded across the dev lifecycle

Ideal organization profiles:

- Engineering-led organizations with high CLI usage
- DevOps/Platform-mature environments
- Cloud-native or automation-heavy teams
- Existing Copilot customers looking to deepen adoption
- GitHub-standardized enterprises expanding AI workflows

How it fits into the narrative

Copilot CLI brings agentic workflows into the terminal so developers can plan, execute, test, and ship without leaving their shell. Customers are evolving from AI assistance (Wave 1) to delegating tasks to agents (Wave 2), and toward multi-agent orchestration (Waves 3–4).

Three capabilities that power the narrative:

- **Agentic orchestration →** developers can plan different approaches with plan mode and use `/fleet` to trigger parallel subagents for repeatable workflows, moving beyond simple chat towards autonomous task execution.
- **Multi-provider choice →** access models from all leading providers (OpenAI, Anthropic, Google) to ensure you always have the best tools for the job. This allows for vendor choice and ensures you'll always be able to use the latest and greatest models in the industry without ever having to switch platforms.
- **Persistent context →** stay in flow without re-explain intent. Copilot remembers your architecture, decisions, and goals across long-running tasks, ensuring work remains coherent without rebuilding context.

30-second elevator pitch

Copilot CLI brings agentic workflows directly into the terminal – but the real unlock is /fleet.

With /fleet, teams can orchestrate parallel subagents or run multiple models at once, turning complex, manual work into repeatable agentic workflows. Instead of one-off assistance, organizations can scale their best engineering patterns across the team.

And because it's included in Copilot Business and Enterprise, admins can confidently roll it out under standard GA terms – no extra billing or complexity.

60-second elevator pitch

Copilot CLI extends AI beyond the IDE and brings Copilot directly into the terminal, so developers can plan, execute, test, and ship without leaving their shell.

Instead of using AI one step at a time, /fleet allows teams to orchestrate parallel subagents or run multiple models simultaneously. That means complex, high-effort work can be transformed into repeatable agentic workflows – scaling expertise across the organization and improving consistency.

Developers stay in flow within the CLI, maintain context across multi-step tasks, and access leading models from providers like OpenAI, Anthropic, and Google – all through a single governed interface.

And because this is included in Copilot Business and Enterprise and now generally available, organizations can adopt and scale it with confidence.

Discovery questions

1. Do you want to explore ways to increase AI usage for your developers?
2. Where do your developers spend the most time outside the IDE, especially in terminal loops (tests, builds, logs, scripts, git)?
3. What are the top recurring "time sinks" where engineers repeatedly re-explain context or redo steps (debugging, refactors, CI failures)?
4. Are you standardizing on one AI model, or do different teams prefer different models for different tasks?
5. If you could templatize one repeatable workflow for every engineer, what would it be (triage, tests, refactors, dependency updates, PR hygiene)?
6. What does "good governance" look like for AI usage in your dev workflow today?

Objection handling

"We already have Copilot in the IDE."
Copilot CLI complements IDE usage. Many high-friction tasks happen in the terminal (build/test loops, git, scripts, debugging logs). CLI keeps the workflow in the execution surface and reduces context switching.

"Persistent sessions sound risky."
Persistence is about remembering context so users do not have to restate it. Developers stay in control through preview-before-execution and explicit commands.

“Model choice will create inconsistency.”
Start with a default model for most users and allow multi-model where it adds value. Multi-model support preserves flexibility as models evolve without changing platforms.

“How do we roll this out safely?”
GA status supports enterprise rollout confidence. Pair rollout with a short pilot, recommended starter workflows, and clear usage guidance.

Outreach templates

New customer

Subject options:

- Quick question about terminal workflows
- Closing the innovation gap in your terminal workflows
- Reducing terminal friction for your engineers

Hi ${name},

I work with engineering teams to help close the "execution gap” in terminal-heavy workflows like CI troubleshooting and infrastructure scripting.

While most teams use AI in the IDE, the terminal remains a manual bottleneck. We’re seeing organizations use Copilot CLI to solve this. Abhishek Chaturvedi, Senior Principal Engineer at Docusign, shared that it’s allowed their engineers to stay in their natural workflow with "less friction and more focus" than switching between tools.

The platform ensures your devs always have access to the latest and greatest models (OpenAI, Anthropic, Google) without leaving their environment.

Would you be available for a quick 15-minute call to talk about how this could accelerate your team’s velocity?

Best,
${sender}

Existing customer

Subject options:

- Expanding Copilot beyond the IDE
- Deepening Copilot usage across workflows
- Copilot in the terminal

Hi ${name},

I know your team is already using Copilot in the IDE, but I wanted to make sure you saw that Copilot CLI is now included in your existing licence at no extra cost.

It's a simple way to increase your AI ROI by moving the "last mile" tasks like CI troubleshooting and scripting into the terminal. Abhishek Chaturvedi, Senior Principal Engineer at Docusign, recently noted that for teams who live in the terminal, it acts as a "dependable sidekick," helping developers move through everyday tasks with more focus and less distraction.

Would you be open to a 10-minute call on how to enable this for your org?

Best,
${sender}

Resources

Copilot CLI **FAQ**
Plug & play **customer-facing slides**